

Name: Peram Mahitha

Reg no: 192524136

1. Scenario: A company wants to deploy a secure access system using face recognition in a low-light environment.

Constraints:

A. Limited computational resources (embedded device)

B. Faces may appear at different orientations ($\pm 30^\circ$)

C. Real-time recognition required (< 200 ms per face)

Questions:

1. How would you adapt Eigenface-based recognition to handle varying illumination and orientation?

2. Suggest preprocessing steps to improve recognition under low-light conditions.

3. Discuss strategies to optimize computation for real-time performance on constrained hardware.

1. Adapting Eigenface-based recognition for varying illumination and orientation

Eigenfaces use Principal Component Analysis (PCA) to represent faces using a smaller set of important features. However, normal Eigenfaces are sensitive to changes in lighting and face angle.

To handle the given constraints:

- **Illumination normalization:** Apply histogram equalization or CLAHE before generating Eigenfaces to reduce the effect of different lighting conditions.
- **Orientation handling:** Since faces can rotate up to $\pm 30^\circ$, detect the face and align it using the eyes as reference points. Rotate the face so that the eyes are approximately horizontal.

- **Multiple face templates:** Store Eigenface representations for several orientations, such as -30° , -15° , 0° , $+15^\circ$, $+30^\circ$.
- **Use PCA on normalized faces:** Generate Eigenfaces from illumination-normalized and aligned training images.
- **Distance-based matching:** Compare the input face with stored Eigenface projections using Euclidean distance and select the closest match.

Thus, face alignment and illumination normalization make Eigenface recognition more robust to changes in lighting and orientation.

2. Preprocessing steps for low-light conditions

The following preprocessing steps can improve recognition:

1. **Face detection:** First detect the face using a lightweight detector.
2. **Face alignment:** Detect the eyes and rotate the face to a standard position.
3. **Grayscale conversion:** Convert the image to grayscale to reduce computation.
4. **Noise reduction:** Apply a small Gaussian or median filter to remove noise commonly present in low-light images.
5. **Contrast enhancement:** Use CLAHE (Contrast Limited Adaptive Histogram Equalization) to improve local contrast without excessively amplifying noise.
6. **Gamma correction:** Apply gamma correction to brighten dark facial regions.
7. **Normalization:** Normalize pixel intensity values to reduce differences caused by lighting.
8. **Resize:** Resize the face to a small fixed resolution, such as 64×64 pixels, before Eigenface processing.

Recommended pipeline:

Input Image $\rightarrow$ Face Detection $\rightarrow$ Grayscale $\rightarrow$ Noise Reduction $\rightarrow$ CLAHE/Gamma Correction $\rightarrow$ Face Alignment $\rightarrow$ Resize $\rightarrow$ Eigenface Recognition

3. Strategies to optimize computation for real-time performance

Since the system uses an embedded device and must recognize a face in less than 200 ms, computational efficiency is important.

a. Reduce image size

Use a small input resolution such as 64×64 or 80×80 pixels instead of processing high-resolution images.

b. Use lightweight face detection

Choose a computationally efficient face detector instead of a large deep-learning model.

c. Reduce Eigenface dimensions

PCA can reduce hundreds or thousands of pixel features to a small number of principal components, for example 20–50 Eigenfaces.

d. Precompute training data

Eigenfaces and the projected training-face vectors should be calculated offline. The embedded device only needs to project the new face and compare it with stored vectors.

e. Use efficient distance calculation

Use Euclidean distance between the input projection and stored face projections. Avoid unnecessary calculations.

f. Limit orientation templates

Instead of storing many rotated versions, use a few representative orientations such as -30° , -15° , 0° , $+15^\circ$, $+30^\circ$.

g. Use integer/fixed-point operations where possible

On hardware with limited processing capability, replacing expensive floating-point operations with optimized or fixed-point calculations can improve speed.

h. Process only the face region

Do not process the entire camera frame. First detect the face and perform recognition only on the cropped face region.

Conclusion

The system can satisfy the constraints by combining illumination normalization, face alignment, lightweight preprocessing, PCA-based dimensionality reduction, and precomputed Eigenface models. Using a small image size and lightweight algorithms reduces computational load, helping the embedded device achieve the required real-time recognition time of less than 200 ms per face.

2. Scenario: You are designing an AI system to automatically organize a large personal photo collection by people and events.

Constraints:

A. Millions of images with inconsistent resolution and quality

B. Multiple photos per person in different poses

C. Privacy considerations: all processing must occur locally

Questions:

1. How would you use face recognition and clustering to group photos of the same individual?

2. Propose a method to detect and separate images with occlusions or partial faces.

3. Discuss trade-offs between accuracy and computational efficiency for local processing.

1. Using face recognition and clustering to group photos of the same individual

The system can combine face detection, face recognition, and clustering to automatically organize photos.

Steps:

1. Face Detection: Detect all faces present in each image using a face detection algorithm.
2. Face Preprocessing: Crop the detected faces and resize them to a common resolution. Normalize brightness and contrast to handle different image qualities.
3. Feature Extraction: Convert each face into a numerical face embedding, which represents important facial characteristics.

4. **Similarity Calculation:** Compare embeddings using a distance measure such as cosine similarity or Euclidean distance.
5. **Clustering:** Group similar face embeddings using algorithms such as DBSCAN or K-Means.
6. **Person Identification:** Each cluster represents a likely individual. If the person's identity is known, the cluster can be assigned their name.
7. **Multiple Poses:** Since the same person may appear from different angles, multiple photos of the person are included in the same cluster when their facial embeddings are sufficiently similar.

Example:

Photos → Face Detection → Face Embeddings → Similarity Calculation → Clustering → Person-wise Albums

DBSCAN can be particularly useful because it can identify outliers, such as incorrectly detected faces or unusual faces that do not belong to any person.

2. Detecting and separating images with occlusions or partial faces

Some photos may contain faces that are partially hidden by sunglasses, masks, hands, hair, objects, or image boundaries.

A suitable method is:

1. **Face Detection:** Detect the face and obtain its bounding box.
2. **Face Quality Check:** Measure the visible area, sharpness, brightness, and resolution of the detected face.
3. **Landmark Detection:** Detect facial landmarks such as eyes, nose, and mouth.
4. **Occlusion Detection:** If important landmarks are missing or the visible face area is too small, classify the face as occluded/partial.
5. **Confidence Threshold:** Use the face detector's confidence score to reject unreliable detections.
6. **Separate Storage:** Place low-quality, heavily occluded, or partial faces into a separate "Unknown/Partial Faces" category rather than incorrectly assigning them to a person.

7. **Reprocessing:** If a better-quality image of the same person is available, it can be used as a reference for future matching.

Example categories:

- Person A
- Person B
- Person C
- Unknown Faces
- Partial/Occluded Faces

This prevents the system from incorrectly grouping an unrelated or partially visible face with the wrong person.

3. Trade-offs between accuracy and computational efficiency for local processing

Since the photos must be processed locally, the system cannot depend on cloud-based processing. Therefore, memory, CPU/GPU usage, storage, and processing time must be considered.

Approach	Accuracy	Computational Cost
High-resolution images	High	High
Low-resolution images	Moderate	Low
Large deep-learning face model	Very high	High
Lightweight face model	Good	Low
More face embeddings	Higher	More storage/computation
Fewer embeddings	Lower	Less storage/computation
Complex clustering	Higher	Higher

Approach	Accuracy	Computational Cost
Simple clustering	Moderate	Lower

Methods to improve efficiency

- Resize images before processing.
- Use a lightweight face detection and recognition model suitable for local devices.
- Generate face embeddings only once and store them locally.
- Use batch processing instead of repeatedly processing the same images.
- Use GPU acceleration if the local computer has a GPU.
- Use clustering algorithms efficiently and remove obvious duplicate images.
- Perform detailed recognition only on images where face detection has high confidence.
- Store only the necessary face embeddings instead of keeping multiple copies of processed images.

Conclusion

A practical system would use face detection → preprocessing → face embeddings → clustering → person-wise organization, while placing uncertain or occluded faces into a separate category. For local processing, a lightweight recognition model with moderate image resolution provides a good balance between accuracy, processing speed, memory usage, and privacy. Since all processing happens locally, the user's photos and facial data do not need to be uploaded to a cloud service.

3. Scenario: A smart city surveillance system must detect moving objects and track pedestrians across multiple cameras.

Constraints:

A. Crowded scenes with frequent occlusion

B. Limited network bandwidth between cameras

C. Real-time processing requirement

Questions:

1. How would you design a foreground–background separation algorithm to minimize false positives in crowded scenes?
2. Explain how particle filters can be used to handle occlusion during tracking.
3. Propose an efficient multi-camera strategy for consistent pedestrian identification across viewpoints.

1. Foreground–Background Separation in Crowded Scenes

Foreground–background separation is used to identify moving objects while ignoring static parts of the scene. In crowded environments, simple background subtraction can produce many false positives, so a more robust approach is required.

Proposed method:

1. **Background Modelling:** Create a background model using several initial frames instead of a single frame.
2. **Adaptive Background Update:** Continuously update the background model so that gradual changes in lighting and environment do not appear as foreground objects.
3. **Shadow Removal:** Detect and remove shadows because shadows can be incorrectly classified as moving objects.
4. **Noise Filtering:** Apply morphological operations such as erosion and dilation to remove small noisy regions.
5. **Connected Component Analysis:** Identify connected foreground regions and discard very small regions.
6. **Object Detection:** Combine background subtraction with an object detector to confirm whether a foreground region is actually a pedestrian.
7. **Temporal Consistency:** Consider an object as a valid moving object only if it appears consistently across multiple frames.

Flow:

Video Frame → Background Model → Background Subtraction → Shadow Removal → Noise Filtering → Object Detection → Pedestrian

This combination reduces false detections caused by shadows, lighting changes, and background noise.

2. Using Particle Filters to Handle Occlusion

A particle filter is useful for tracking pedestrians when their appearance temporarily becomes hidden by another person or object.

Instead of representing a pedestrian's position with only one point, the particle filter maintains many possible states called particles.

Working:

1. **Initialization:** Generate particles around the pedestrian's initial position.
2. **Prediction:** Predict where the pedestrian is likely to move in the next frame.
3. **Observation:** Compare the predicted positions with the available image information.
4. **Weighting:** Give higher weights to particles that better match the pedestrian's appearance and motion.
5. **Resampling:** Select the particles with higher probabilities and generate a new set of particles.
6. **Occlusion Handling:** If the pedestrian becomes temporarily hidden, visual information becomes unreliable. The particle filter continues predicting the pedestrian's position using its previous motion information.
7. **Recovery:** When the pedestrian becomes visible again, the filter uses the new observation to correct the predicted position.

Example:

If Person A walks behind Person B for several frames, the particle filter can continue predicting Person A's position. When Person A becomes visible again, the tracker can re-estimate the correct position.

Thus, particle filters provide robust tracking during temporary occlusions.

3. Efficient Multi-Camera Strategy for Consistent Pedestrian Identification

For multiple cameras, sending complete video streams between cameras would require a large amount of network bandwidth. Therefore, the system should perform most processing locally at each camera.

Proposed architecture

At each camera:

Camera → Local Object Detection → Tracking → Feature Extraction → Send Metadata

Instead of transmitting complete video, each camera sends only:

- Pedestrian ID
- Timestamp
- Camera ID
- Bounding-box coordinates
- Movement direction
- Compact appearance features/embedding

A central server or edge coordinator can then match pedestrians across cameras.

Cross-camera identification

1. Detect and track a pedestrian locally.
2. Extract an appearance feature/embedding from the pedestrian.
3. Assign a temporary local ID, such as Camera1-P12.
4. Send the compact ID and feature information instead of the entire video.
5. When the pedestrian appears in another camera, extract its features.
6. Compare the features with previously stored features using a similarity measure.
7. If the similarity is above a predefined threshold and the timing/location is reasonable, assign the same global identity.

For example:

Camera 1: Person P12 → Feature A

↓

Camera 2: Feature A' → Similarity Check

↓

Global ID: Person 5

Handling limited bandwidth

To reduce network usage:

- Perform object detection and tracking at the camera/edge device.
- Transmit only metadata and compact feature vectors.
- Send full video only when required for verification.
- Compress transmitted features and metadata.
- Use event-based communication rather than continuously transmitting video.

Conclusion

A robust smart-city surveillance system can combine adaptive background subtraction, object detection, particle-filter tracking, and appearance-based cross-camera matching. Local processing reduces network bandwidth requirements, while particle filters help maintain tracking during occlusion. Sending compact pedestrian features rather than complete video makes the system more suitable for real-time multi-camera surveillance.

4. Scenario: A self-driving car must navigate urban streets while detecting lanes, road signs, and pedestrians.

Constraints:

A. Variable weather and lighting conditions (rain, night)

B. Real-time decision-making (≤ 50 ms per frame)

C. High-density traffic with moving pedestrians

Questions:

1. How would you implement road detection robust to shadows, rain, and nighttime illumination?

2. Propose a pipeline to simultaneously detect road signs and pedestrians efficiently.

3. Discuss the trade-offs between deep learning-based detectors and classical computer vision methods under real-time constraints.

1. Road detection robust to shadows, rain, and nighttime illumination

For a self-driving car, road detection should work under different lighting and weather conditions. A combination of image preprocessing, color-based segmentation, edge detection, and adaptive models can be used.

Proposed approach:

1. **Image preprocessing:** Convert the camera image to a suitable color space such as HSV or HLS, which can separate brightness from colour information.
2. **Illumination correction:** Use techniques such as gamma correction and CLAHE to improve visibility in dark scenes.
3. **Shadow reduction:** Use color and intensity information to distinguish road regions from dark shadows.
4. **Rain handling:** Apply noise reduction and temporal filtering to reduce the effect of raindrops and reflections.
5. **Lane detection:** Use edge detection followed by a region of interest (ROI) to focus only on the road area.
6. **Adaptive thresholding:** Use thresholds that change according to the current lighting conditions rather than fixed thresholds.
7. **Temporal information:** Compare consecutive frames to maintain stable road and lane detection.

Basic flow:

Camera Frame → Preprocessing → Shadow/Rain Reduction → ROI Selection → Road/Lane Detection → Tracking

For difficult nighttime and rainy conditions, a lightweight trained segmentation model can be added to improve reliability.

2. Pipeline to simultaneously detect road signs and pedestrians efficiently

Since the vehicle needs to make decisions in real time, both road signs and pedestrians should be detected efficiently.

Proposed pipeline:

Step 1 – Image acquisition

Capture frames from the vehicle's camera.

Step 2 – Preprocessing

Resize the frame and perform basic brightness/noise correction.

Step 3 – Object detection

Use a lightweight object-detection model to detect:

- Pedestrians
- Traffic signs
- Vehicles
- Other important road objects

Step 4 – Classification

For detected traffic signs, classify them as:

- Stop
- Speed limit
- No entry
- Warning signs, etc.

Step 5 – Tracking

Track detected pedestrians and signs across consecutive frames so that the system does not need to detect everything from scratch in every frame.

Step 6 – Decision-making

Combine the detected objects with lane and road information to determine the vehicle's next action.

Efficient pipeline

Camera → Resize/Preprocess → Lightweight Detector → Sign & Pedestrian Detection → Tracking → Decision Module

To achieve ≤ 50 ms per frame, the system can use a lightweight neural network, GPU/accelerator support, reduced image resolution, and frame-to-frame tracking.

3. Trade-offs between Deep Learning and Classical Computer Vision

Feature	Deep Learning	Classical Computer Vision
Accuracy	Generally high	Moderate
Complex environments	Better	Limited
Rain/night conditions	Can be robust if trained properly	Often sensitive
Computational requirement	High	Low
Training required	Yes	Usually no
Adaptability	High	Lower
Real-time performance	May require GPU/accelerator	Usually faster
Development complexity	Higher	Lower

Deep Learning Methods

Examples include lightweight object-detection networks such as YOLO-type models.

Advantages:

- Can detect pedestrians and road signs accurately.
- Handles different object shapes and viewpoints.

- Can learn complex features automatically.
- Better suited for crowded traffic scenes.

Disadvantages:

- Requires significant computational resources.
- Requires training data representing rain, night, shadows, etc.
- May be difficult to achieve 50 ms/frame on low-power hardware without optimization.

Classical Computer Vision

Methods include:

- Canny edge detection
- Hough Transform
- Colour segmentation
- Thresholding
- Contour detection
- Haar-like features

Advantages:

- Computationally inexpensive.
- Easier to implement.
- Suitable for simple lane-detection tasks.
- Can provide very fast processing on limited hardware.

Disadvantages:

- Sensitive to lighting, shadows, rain, and occlusion.
- Difficult to reliably detect pedestrians and signs in complex traffic.
- Requires manually designed rules and thresholds.

Conclusion

For an autonomous vehicle, a hybrid approach is often suitable. Classical computer vision can be used for computationally simple tasks such as basic lane detection, while a lightweight deep-learning detector can handle pedestrians and road signs. Model quantization, lower image resolution, GPU/AI accelerator support, and object tracking can help achieve the ≤ 50 ms per-frame real-time requirement while maintaining good detection accuracy.

5. Scenario: An AR system for surgical assistance must overlay segmented organs in real time.

Constraints:

- A. High-resolution 3D medical images (CT/MRI)**
- B. Latency must be < 100 ms for surgeon interaction**
- C. Limited GPU memory on wearable AR headset**

Questions:

- 1. How would you implement real-time organ segmentation under these memory and latency constraints?**
- 2. Discuss methods to integrate segmentation results accurately in AR/MR overlays.**
- 3. Suggest strategies to reduce computational load while maintaining segmentation accuracy.**

1. Real-time organ segmentation under memory and latency constraints

The system must process high-resolution CT/MRI data while keeping latency below 100 ms and using limited GPU memory. A lightweight segmentation model with optimized 3D processing can be used.

Proposed approach:

- 1. Preprocess CT/MRI images:** Normalize intensity values and remove unnecessary noise.
- 2. Region of Interest (ROI):** Instead of processing the complete 3D scan, identify and process only the region containing the target organ.

3. **Downsample the volume:** Reduce the resolution while preserving important anatomical information.
4. **Use a lightweight segmentation model:** A compact **3D U-Net** or **2D/2.5D U-Net** can be used instead of a large 3D network.
5. **Patch-based processing:** Divide the large 3D volume into smaller overlapping patches and process them individually.
6. **GPU memory optimization:** Use reduced-precision computation such as FP16 and avoid storing unnecessary intermediate feature maps.
7. **Temporal consistency:** For live video, update the segmentation only when necessary and track the previously segmented organ between frames.

Basic pipeline:

CT/MRI → Preprocessing → ROI Selection → Downsampling/Patch Extraction → Lightweight Segmentation → Organ Mask → AR Overlay

This reduces the amount of data processed at each stage and helps achieve the <100 ms latency requirement.

2. Integrating segmentation results accurately into AR/MR overlays

The segmented organ must appear at the correct position and orientation in the real surgical environment. This requires accurate registration between the medical image and the patient's physical anatomy.

Steps:

1. **Image-to-patient registration:** Align the CT/MRI coordinate system with the patient's actual body position.
2. **Tracking:** Use cameras, depth sensors, or markers to continuously track the patient's position and the headset.
3. **Coordinate transformation:** Convert the 3D segmentation coordinates into the AR headset's coordinate system.
4. **Pose estimation:** Continuously estimate the position and orientation of the patient and headset.

5. **Overlay rendering:** Render the segmented organ as a semi-transparent 3D model over the corresponding anatomical location.
6. **Continuous correction:** Update the overlay as the patient or headset moves.

Flow:

CT/MRI Segmentation → 3D Organ Model → Registration → Patient/Headset Tracking → Coordinate Transformation → AR Overlay

To improve accuracy, the system should use fiducial markers, depth sensing, or feature-based registration where appropriate.

3. Strategies to reduce computational load while maintaining accuracy

Several optimization techniques can be applied:

a. ROI-based processing

Process only the anatomical region needed for the current surgical task rather than the complete CT/MRI volume.

b. Down sampling

Reduce image resolution before segmentation while retaining important anatomical structures.

c. Lightweight models

Use compact architectures such as lightweight U-Net variants instead of very large segmentation networks.

d. Patch-based segmentation

Process smaller sections of the 3D image rather than loading the entire volume into GPU memory.

e. Model quantization

Convert model parameters from high precision to FP16 or INT8 where accuracy remains acceptable.

f. Knowledge distillation

Train a small student model to reproduce the results of a larger, more accurate teacher model.

g. Hardware acceleration

Use the headset's GPU/NPU and optimized inference frameworks to speed up model execution.

h. Frame skipping and tracking

For continuously changing AR scenes, do not perform complete segmentation on every frame. Perform segmentation periodically and track the organ between updates.

i. Multi-resolution processing

Use high resolution only around important anatomical boundaries and lower resolution for less important regions.

Conclusion

A suitable solution is a lightweight, ROI-based, patch-wise 3D segmentation system combined with FP16/INT8 optimization and hardware acceleration. Accurate 3D registration and real-time tracking are then used to place the segmented organs correctly in the AR environment. These techniques provide a good balance between segmentation accuracy, GPU memory usage, and the required <100 ms interaction latency.